

在过去的一年里，大语言模型（LLM）已经从「能写文章」进化到了「几乎可以参与一切文本交互」的阶段。但有一个领域却显得有些「格格不入」——输入法。我们每天都在用输入法打字，但它似乎还停留在「词频 + 简单语言模型」的时代。

那么问题来了：如果把大语言模型真正「用对地方」，输入法会变成什么样？

这篇文章想介绍一种我认为非常有潜力的方向：**约束解码（Constrained Decoding）**。它不仅能把 LLM 的能力引入输入法，还能避免很多已有方案的根本问题。

一、大语言模型到底在做什么？

很多人已经在用 LLM，但对它的内部机制其实并不清楚。理解这一点，对于理解后面的「约束解码」非常关键。

1. 文本是如何被拆分的？

大语言模型并不是直接处理「字」或「词」，而是处理一种叫做 **token（词元）** 的单位。

比如一句话：

我喜欢吃苹果

在模型内部，可能被切分成：

```
["我", "喜欢", "吃", "苹果"]
```

也可能是：

```
["我", "喜", "欢", "吃", "苹", "果"]
```

甚至更复杂。关键点是：**token** 的划分是模型训练时决定的，不一定和我们直觉中的「字」或「词」一致。

2. 模型是如何「生成文字」的？

本质上，LLM 在做一件事：

根据已有文本，预测下一个 **token** 的概率分布

用数学语言来说，就是：

$$P(t_{i+1} \mid t_1, t_2, \dots, t_i)$$

比如当前上下文是：

我今天很

模型可能给出：

- 开心 (0.35)
- 累 (0.25)
- 高兴 (0.15)
- 忙 (0.10)
- ...

然后我们从中「采样」或者选最大概率的，就得到了下一个 token。

不断重复这个过程，就生成了一整段文本。

二、现有「LLM + 输入法」的两条路

目前已经有一些尝试，把 LLM 用在输入法里。但我认为它们都有一个共同的问题：把 LLM 当成黑箱来用。

1. 用 LLM 做「联想输入」

例如你输入：

我今天去了

输入法可能调用 LLM，给你补全：

- 我今天去了公园散步
- 我今天去了公司开会

这看起来很智能，但问题也很明显：用户很难控制结果。

比如你其实想输入：

我今天去了图书馆

但模型给你的候选可能完全不相关，你只能继续打字，而不是「筛选」。

换句话说：LLM 在「帮你写」，而不是「帮你输入」。

2. 训练一个「拼音 → 汉字」的模型

另一类方法是：直接在一个基础大语言模型上微调，让它能把拼音转换成汉字，比如：

```
wo jin tian → 我今天
```

这看起来很合理，但问题在于：

👉 每种输入方案都要单独训练模型

- 全拼要训练一个模型
- 双拼要训练一个模型
- 五笔要训练一个模型

而且一旦编码规则变化，模型基本就要重训。如果换了基础模型，也要重新训练。

这在工程上是非常昂贵的，也不够灵活。

三、约束解码：把 LLM 「驯服」的关键

约束解码其实不是一个新概念，它已经在很多场景中使用，比如：

- 生成合法 JSON
- 自动补全代码（必须符合语法）
- 按模板生成文本

核心思想非常简单：

在生成过程中，对模型的输出施加约束，只允许合法结果出现。

在输入法中的应用

我们可以这样看输入法问题：

- 用户输入的是一种「编码」（比如拼音）
- 我们希望输出符合这个编码的汉字

那为什么不直接把编码作为约束条件，限制 LLM 的生成呢？

例如用户输入 wo，我们只允许模型输出：

- 我
- 沃
- 握

而像「你」「他」这样的 token，直接过滤掉。

这样做的两个巨大优势

1. 可控性极强

模型仍然提供「语言理解能力」，但最终结果完全由编码约束筛选。

👉 你输入什么，就只能得到匹配的内容。

2. 完全不需要训练

约束来自编码规则，而不是模型本身：

- 换输入方案？改约束就行
- 换模型？无需重训

👉 输入法和 LLM 完全解耦

这在工程上是一个非常重要的优势。

四、系统是怎么工作的？（用全拼举例）

我们来把整个过程具体展开一下。假设我们使用全拼输入。

为了简化，我们先忽略拼音切分歧义，假设输入已经被分成音节，然后我们总是给出一个和音节数量长度相等的候选词。

Step 1：初始化模型

给模型一个提示词，比如：

请根据以下内容继续写：

或者以用户之前输入的句子为提示词。然后让模型预测下一个 token，得到一个列表：

```
[
  ("我", 0.30),
  ("今天", 0.15),
  ("你", 0.10),
  ...
]
```

Step 2: 用户输入第一个键

用户输入 `w`，我们过滤 `token` 列表，只保留拼音以 `w` 开头的。例如剩下：

```
[
  ("我", 0.30),
  ("我们", 0.05),
  ("为什么", 0.03),
  ("握", 0.01)
]
```

Step 3: 展开路径（关键点）

现在，用户输入的编码只有一个音节，列表中「我」、「握」为一个音节，而「我们」「为什么」多于一个音节。假设用户又按了下一个键，那么编码可能变成两个音节（即简拼，例如 `wm`），也有可能保持一个音节（例如 `wo`）。

所以，为了对这种两个音节的情况做出准备，那么我们需要对只有一个音节的 `token` 提前探索其后续可能性，以便用户下次输入后立刻做出反应。

于是：「我」、「握」都得到了进一步展开，概率列表变成：

```
[
  ("我", 0.30, [("是", 0.50), ("没", 0.10)]),
  ("我们", 0.05),
  ("为什么", 0.03),
  ("握", 0.01, [("手", 0.60), ("住", 0.10)])
]
```

Step 4: 用户继续输入第二个键

此时有两种情况：

1. 用户继续完成当前音节

例如输入 `o`，现在编码为 `wo`。则按照 Step 2 的方法再次过滤，只留拼音以 `wo` 开头的。如果这样做导致有些提前展开的路径被过滤掉了，那么可以补充展开一些。

2. 用户开启新的音节

例如输入 `m`，现在编码为 `wm`。则过滤得到简拼是 `wm` 的，然后按照 Step 3 的方法再次展开。

Step 5: 重复这个过程

每输入一个音节：

👉 先过滤，再生成

最终我们得到一棵候选树，从中选出概率最高的若干路径作为候选。

五、性能是否可行？

这个问题其实是很多人最关心的。本质上，用户每按下一个键，系统总是先过滤得到候选列表，然后在用户下一次按键之前完成展开。我们可以简单估算一下：

- 过滤操作：几乎可以忽略
- 生成一个 token：不超过 10 ms（假设内存带宽 100 GB/s，加载 1B 小模型的参数需要 0.01 秒，计算时间相对于访存时间可以忽略不计）

假设用户打字速度不超过峰值：10 键/秒（100 ms/键），那么系统只要限制展开的数量在 10 个以下，则完全是可行的，甚至有富余时间。

六、总结：输入法可能迎来一次范式转变

我们可以这样总结：

传统输入法是：

| 编码 → 查表 → 排序

而基于约束解码的输入法是：

| LLM（预测）+ 编码（约束）→ 动态生成候选

它结合了两者的优点：

- 编码提供**精确控制**
- LLM 提供上下文理解

更重要的是：

👉 它不依赖任何特定输入方案，也不需要重新训练模型

这意味着，我们第一次有机会设计一种真正「AI 原生」的输入法。

参考实现

如果你对这个方向感兴趣，可以看看这个项目：

👉 <https://github.com/xushengfeng/lime>